

Universidad Pontificia Comillas

ETSI-ICAI

Laboratorio Robots autónomos

EXPLORACIÓN: SEGUIMIENTO DE PARED

Lydia Ruiz Martínez, Pablo Tuñón Laguna, Jorge Vančo Sampedro



December 2024

1. Incluya en el informe un diagrama de `rqt_graph` que permita entender el flujo de información entre los dos nodos que ha programado en la práctica y explíquelo brevemente.

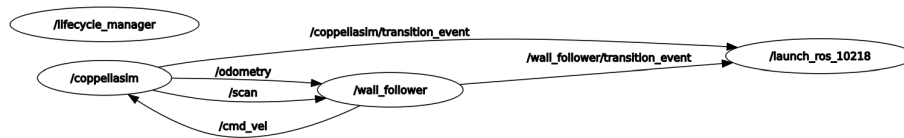


Figura 1: Gráfico de nodos y temas en ROS2 usando `rqt_graph`, autoría propia

En primer lugar se encuentra el `lifecycle_manager`, que se encarga de la correcta inicialización de todos los nodos en el orden adecuado.

El nodo de `/coppelasim` es el nodo del simulador. Este publica la odometría y las detecciones del LiDAR en los topics `/odometry` y `/scan` respectivamente.

El nodo `/wall_follower` está suscrito a estos topics y recibe la información. Este se encarga de calcular los comandos de velocidad lineal y angular y publicarlos por el topic `/cmd_vel`. Los mensajes de este topic son recibidos por el nodo `coppelasim` para ejecutarlos en el simulador.

En el caso real, se sustituiría el nodo `/coppelasim` por el nodo del robot, que tendría la misma función pero movería el robot real.

2. Indique todos los cambios que hay que realizar en el código de simulación para poder ejecutarlo en el TurtleBot3 Burger real. La mayoría se han ido explicando a lo largo del enunciado de esta práctica, pero también tendrá que rescatar información de la anterior.

Primero se necesita crear un nodo que esté suscrito al topic `/odom`. Este nodo se encarga de publicar la odometría correcta en el topic `/odometry`, ya que el robot envía información de información, mientras que nos es de mayor interés la odometría de la velocidad.

El nodo `/odometry_node` obtiene la posición y orientación de la odometría del robot. Calcula la velocidad lineal aplicando el teorema de Pitágoras y dividiendo entre el tiempo entre las mediciones. También es necesario convertir la orientación a ángulos Eulerianos, pues viene dada en cuaterniones. La velocidad angular se obtiene dividiendo la diferencia de mediciones consecutivas de ángulo entre la diferencia de tiempo entre mediciones.

```
def odom_callback(self, msg: Odometry):
```

```

    x = msg.pose.pose.position.x
    y = msg.pose.pose.position.y
    quat_w = msg.pose.pose.orientation.w
    quat_x = msg.pose.pose.orientation.x

```

```
quat_y = msg.pose.pose.orientation.y
quat_z = msg.pose.pose.orientation.z

_, _, th_h = quat2euler((quat_w, quat_x, quat_y, quat_z))

t1 = Time.from_msg(msg.header.stamp).nanoseconds

if self.prev_odom[0] == None:
    self.prev_odom[0] = x
    self.prev_odom[1] = y
    self.prev_odom[2] = th_h
    self.prev_odom[3] = t1

else:
    z_v = np.sqrt((x - self.prev_odom[0])**2 + (y -
        self.prev_odom[1])**2) / ((t1 - self.prev_odom[2]) * 1e-9)
    z_w = (th_h - self.prev_odom[1]) / ((t1 - self.prev_odom[2]) *
        1e-9)
    self._publish_odometry(z_v, z_w)
```

Además, se ha de cambiar el signo de la velocidad angular que se publica porque el sistema de referencia es distinto en el robot que en el simulador, pues el robot tiene el mismo que se suele poner en los drones.

```
def _publish_velocity_commands(self, v: float, w: float) -> None:
    """Publishes velocity commands in a geometry_msgs.msg.TwistStamped
    message.

    Args:
        v: Linear velocity command [m/s].
        w: Angular velocity command [rad/s].

    """
    cmd_vel_msg = Twist()
    cmd_vel_msg.linear.x = v
    cmd_vel_msg.angular.z = -1.0*w # + in the simulation
    self._cmd_vel_publisher.publish(cmd_vel_msg)
```

También se tiene que reducir el valor de retardo (slop) en el objeto ApproximateTimeSynchronizer. En este caso, lo hemos puesto a 0,25.

```
ts = message_filters.ApproximateTimeSynchronizer(
    self._subscribers, queue_size=10, slop=0.25
)
```

Por último, se tiene que cambiar el tipo de mensaje que se publica en el topic /cmd_vel. En simulación se tiene TwistStamped, pero es necesario cambiarlo a Twist.

Twist no tiene la cabecera con la marca de tiempo ya que solo es necesario para la sincronización en el simulador.

```
self._cmd_vel_publisher = self.create_publisher(Twist, "/cmd_vel", 10)
```
